A REPORT OF SEMESTER INDUSTRIAL TRAINING

at

CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING (C-DAC), MOHALI

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

(Computer Science and Engineering)

JANUARY 2026 - JULY 2026

SUBMITTED BY:
NAME: SIMARPREET SINGH
UNIVERSITY ROLL NO: 2224288

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
I.K. GUJRAL PUNJAB TECHNICAL UNIVERSITY KAPURTHALA

CERTIFICATE BY COMPANY

This is to certify that the project report entitled **"Advanced AI Interview System"** is a bonafide work carried out by **SIMARPREET SINGH** (Roll No: **2224288**) in partial fulfillment of the requirements for the award of Bachelor of Technology degree in **Computer Science and Engineering** from **I.K. GUJRAL PUNJAB TECHNICAL UNIVERSITY KAPURTHALA**.

This work was carried out under our supervision and guidance during his Semester Industrial Training from January 2026 to July 2026 at the **Centre for Development of Advanced Computing (C-DAC), Mohali**.

During this six-month internship period, the candidate demonstrated exceptional programming ability, deep technical insight, and strong professional behavior. The software components developed by him—specifically the proctoring modules and the dual-synchronization database systems—have been thoroughly tested and deployed. We find the project report to be an authentic record of the trainee's individual work.

Project Mentor / Guide CDAC Mohali	**Head of Division** CDAC Mohali

CANDIDATE'S DECLARATION

I, **SIMARPREET SINGH**, hereby declare that the training work presented in this report titled **"Advanced AI Interview System"** submitted to the Department of Computer Science and Engineering at **I.K. GUJRAL PUNJAB TECHNICAL UNIVERSITY KAPURTHALA** is an authentic record of my individual semester industrial training work carried out at **CDAC Mohali** under the supervision of my industrial mentors.

This work has not been submitted in part or full to any other university or institute for the award of any other degree or diploma. All the modules, integrations, and database schemas discussed in this report were developed and verified by me under the guidance and support of the technical staff at CDAC Mohali during the training period from January 2026 to July 2026.

Signature of the Student:

_____

SIMARPREET SINGH
Roll No: 2224288

ABSTRACT

Automated mock evaluation platforms are rapidly replacing traditional manual screening rounds in engineering recruitment. The **Advanced AI Interview System** is a state-of-the-art software simulator designed to streamline candidate vetting by implementing client-side visual proctoring, real-time emotion mapping, automated Whisper-based speech transcription, and LLM-powered response grading.

This project report documents the implementation of the system's core engines, including: (1) a multi-tab client-side webcam proctor that logs window-blurring and look-away events; (2) an asynchronous parser that extracts qualifications, branches, and database technologies from uploaded candidate resumes; (3) a dynamic semantic question retrieval subsystem; and (4) a dual-database schema backing local SQLite development databases and Neon PostgreSQL production environments with real-time transactional synchronization scripts.

In addition, the report outlines the testing and QA plan designed to validate parsing correctness, verify LLM fallbacks to local TF-IDF Cosine Similarity grading under network dropouts, and ensure data integrity during simultaneous transactional writes. The operational results confirm that the platform is scalable, low-latency, and suitable for high-throughput engineering recruitments.

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to the **Centre for Development of Advanced Computing (C-DAC), Mohali** for giving me the opportunity to complete my semester industrial training and work on this state-of-the-art software development project.

I am extremely grateful to my industrial mentors and guides at C-DAC Mohali for their invaluable suggestions, constant encouragement, and support during the technical implementation of the Advanced AI Interview System. Their hands-on expertise in backend systems and software engineering practices has greatly enhanced my design thinking and coding standards.

I also extend my sincere thanks to the Head of the Department and all the faculty members of the Department of Computer Science and Engineering at **I.K. GUJRAL PUNJAB TECHNICAL UNIVERSITY KAPURTHALA** for providing the academic foundation and coordinating my semester internship program.

Lastly, I want to thank my parents, family, and peers for their constant support, motivation, and constructive feedback throughout this training project.

SIMARPREET SINGH

ABOUT THE COMPANY

The **Centre for Development of Advanced Computing (C-DAC)** is the premier R&D; organization of the Ministry of Electronics and Information Technology (MeitY) for carrying out R&D; in IT, Electronics and associated areas. C-DAC has emerged as a key nation-building institution, delivering high-performance computing, multilingual technology, cyber security, software engineering, and professional training solutions.

C-DAC Mohali, one of the leading resource centers, focuses on various advanced fields including Health Informatics, Artificial Intelligence, Computer Vision, Embedded Systems, and Information Security. C-DAC Mohali is committed to research and development of software applications that address national requirements and cater to the training demands of engineering professionals and students alike.

The center promotes a project-centric and hands-on learning model, enabling interns to participate in real-world software engineering pipelines. By providing access to high-compute clusters, state-of-the-art software tools, and guidance from senior scientists, C-DAC Mohali prepares trainees to design robust, secure, and enterprise-grade software products in line with industry demands.

TABLE OF CONTENTS

TABLE OF CONTENTS (Continued)

LIST OF FIGURES & TABLES

List of Figures:

List of Tables:

DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Term	Definition
API	Application Programming Interface, standard for inter-module integration.
FastAPI	High-performance asynchronous Python web framework built on Starlette and Pydantic.
LLM	Large Language Model, used for grading candidate descriptive transcript answers.
VAD	Voice Activity Detection, algorithm that checks for sound energy anomalies.
JSON	JavaScript Object Notation, standard format for payload transactions.
ORM	Object-Relational Mapping, simplifies schema transactions via models.
JWT	JSON Web Token, compact and self-contained security token standard.
WebRTC	Web Real-Time Communication, client-side capture pipeline standard.
TF-IDF	Term Frequency-Inverse Document Frequency, numerical statistic for grading.
ETL	Extract, Transform, Load, database sync pipeline process model.

CHAPTER 1: INTRODUCTION

1.1 Project Context and Background

Modern corporate recruitment is bottlenecked by the heavy cost and time constraints of human technical interviewing. In the technology sector, companies receive hundreds of resumes for a single engineering opening. Traditional pipelines require human reviewers to parse applications and coordinate preliminary phone-screens. This manual approach is highly inefficient, subjective, and difficult to scale in high-volume situations.

Semester industrial training provides a unique opportunity to address these challenges by building automated, data-driven software platforms. The project documented in this report—the **Advanced AI Interview System**—is a multi-modal mock interview room simulator. It aims to bridge the screening gap by automating candidate resume parsing, conducting interactive audio-video mock screenings, and providing recruiters with deep analytics dossiers, saving time and resources in early screening stages.

1.2 Purpose and Scope of Mock Interview Simulators

The primary purpose of the Advanced AI Interview System is to provide candidates with a highly realistic, interactive, and proctored simulation environment to evaluate their technical competence and communication skills. The system models a live interviewer by dynamically presenting categorized questions, tracking responses, and providing detailed, prompt grading criteria without requiring manual scheduling or intervention.

The scope of this training project encompasses the full-stack development of: (1) an administrative panel that enables recruitments managers to seed mock candidates, calibrate domain questions, view candidate transcripts, inspect scoring metrics, and track webcam proctor logs; (2) a client-side simulation chamber integrating media capture APIs, TensorFlow-based facial monitors, and speech recognition; and (3) a highly resilient backend API processing files, handling fallback scoring, and synchronizing relational candidate datasets.

1.3 Need and Importance of AI in Recruitment

Automating early-stage candidate screening has become a core objective for modern, data-driven HR organizations. Traditional veting methods suffer from subjective human grading biases, inconsistency in evaluation criteria across interviewers, and long turnaround times that cause companies to lose top talent to competitors. An AI-powered mock room resolves these issues by providing centralized, uniform grading metrics that assess candidates on standardized parameters.

Furthermore, the system provides high scalability. Hundreds of applicants can perform interviews concurrently, eliminating calendar coordination delays. By automating candidate evaluation, recruiters can skip initial screening phone calls and focus their attention solely on candidates who meet objective, verified scoring thresholds. This significantly improves recruitment throughput while maintaining consistency and standardizing quality assessment.

1.4 Problem Statement and Challenges in Proctoring

While automated interviews solve scheduling and throughput issues, they introduce a major security challenge: verification of test integrity. Without a human proctor present, candidates can easily look away to read answers on secondary screens, switch browser tabs to consult online search engines, or have an unauthorized person assist them off-camera. Ensuring absolute test honesty is critical to making the automated assessment scores meaningful and actionable for recruiter decision-making.

To solve this problem, the Advanced AI Interview System integrates browser tab visibility listeners, webcam state tracking, and real-time eye-focus detection. The system captures tab blurring events, look-away gestures, and face-missing violations, appending timestamped proctoring flags directly to the candidate's database profile. This proctoring data is compiled into recruiter dashboards, warning HR managers of potential integrity violations.

1.5 Objectives of the Project

The semester training project was executed with the following core software development and research objectives:

- **1. Resume Parsing Automation:** Create an intelligent backend parser using pdfplumber and regular expression heuristics to extract emails, phone numbers, and qualifications, and categorize candidate branches (e.g. CSE, Data Science, AIML).
- **2. Multi-Modal Proctoring:** Implement real-time client-side face detection using Face-API.js to identify cheating patterns, gaze drift, and face presence anomalies, logging proctor violations with precise timestamps.
- **3. Dynamic Question Selection:** Build a domain-driven question retriever that matches candidate resume skills with categorized database question pools, preventing duplicate questions across active interview sessions.
- **4. Asynchronous Speech-to-Text:** Configure celery workers and Redis task queues to run OpenAI Whisper transcription pipelines, converting audio recordings to text without blocking the web request-response cycle.

1.6 Organization Profile: CDAC Mohali

The Centre for Development of Advanced Computing (C-DAC) is the premier R&D; organization under the Ministry of Electronics and Information Technology (MeitY). C-DAC Mohali was established to promote industrial R&D; in core technology domains. The center operates through specialized divisions focusing on Cyber Security, Health Informatics, Artificial Intelligence, and Computer Vision.

During this six-month semester training, the candidate worked inside the Artificial Intelligence & Computer Vision group, participating in project sprints and building robust software pipelines. C-DAC Mohali provides state-of-the-art facilities and technical mentoring, allowing trainees to apply theoretical algorithms to real-world deployment stacks, undergo rigorous code reviews, and understand modern software architecture paradigms.

1.7 Organization and Structure of Report

This industrial training report has been organized into six core chapters to detail the architecture and implementation of the Advanced AI Interview System. Chapter 1 introduces the project background, objectives, and profile of the training organization. Chapter 2 presents the technology survey, outlining the frontend and backend frameworks, databases, and AI libraries integrated into the platform.

Chapter 3 covers requirements analysis, detailed Software Requirements Specifications (SRS), and system database designs. Chapter 4 provides the core technical implementation, featuring database migration scripts, resume parsers, proctoring algorithms, and API handlers. Chapter 5 details verification protocols, unit tests, and LLM fallback assessments. Finally, Chapter 6 provides the conclusion, learning outcomes, and technical roadmaps for future enhancements.

CHAPTER 2: FIELD OF TRAINING & TECH SURVEY

2.1 Overview of Field of Training

The field of training encompasses Full-Stack Web Engineering, Artificial Intelligence, and Computer Vision. As part of the AI & Computer Vision internship at C-DAC Mohali, the candidate worked extensively with web protocols, media streaming APIs, client-side machine learning execution, and relational data management. Building a mock interview room requires integrating backend database endpoints with real-time browser-based detectors.

The technology survey conducted during training focused on selecting libraries that minimize runtime latency and external api costs. The system was designed using Python's FastAPI to leverage asynchronous event loops, while client-side TensorFlow execution via Face-API.js was chosen to handle eye tracking and emotion detection on the client browser. This approach avoids sending heavy video streams to the backend, drastically reducing server load and bandwidth utilization.

2.2 Frontend Stack: HTML, CSS, JavaScript

The client-side interface is developed using standard web technologies: HTML5, CSS3, and modern JavaScript (ES6+). Vanilla CSS was chosen for styling to retain absolute control over layout flow and responsiveness. The design system utilizes custom CSS variables, subtle CSS transitions, and glassmorphism styling to create a premium, immersive dark-themed user experience that mimics a desktop client.

JavaScript handles webcam media capture using the browser's `navigator.mediaDevices.getUserMedia` API. It captures audio-video data in chunks, manages state machines for dynamic question rendering, and controls browser focus listeners to log cheating warnings. The client-side dashboard coordinates data fetching via Fetch APIs and dynamically renders candidate performance graphs leveraging Highcharts JS, ensuring interactive and fast-loading charts.

2.3 Client-Side ML Models: Face-API.js

For real-time proctoring and behavioral analysis, the browser frontend utilizes **Face-API.js**, which runs pre-trained TensorFlow models directly in the browser client. This setup performs two critical functions: (1) extracting emotion matrices from the candidate's facial features (mapping confidence, neutrality, sadness, and fear), and (2) monitoring head orientation and eye focus by tracking facial landmark coordinate vectors in real-time.

Running AI models client-side has major security and architectural advantages. First, candidate webcam video remains entirely inside the local browser memory during proctoring, protecting privacy. Second, it eliminates the need to upload high-definition video feeds to the server for AI processing. This keeps compute costs low and allows the backend to operate on thin, low-cost virtual containers without requiring expensive GPU hosting nodes.

2.4 Backend Server Framework: FastAPI

The backend server is engineered using Python's **FastAPI** framework. FastAPI was selected because it leverages asynchronous ASGI specification (built on Uvicorn and Starlette), delivering execution speeds comparable to Node.js and Go. FastAPI's native support for coroutine asynchronous functions allows the server to manage thousands of concurrent candidate requests, SSE streams, and file uploads without blocking the main event execution loop.

FastAPI also streamlines development by automatically parsing request models using Pydantic, enforcing database transactions, and generating interactive OpenAPI documentation (Swagger UI). The backend configures CORS middlewares to allow communication from Vercel static frontends, exposes secure routes for resume uploads, and integrates Celery background task managers to queue complex audio transcriptions and grading requests to separate worker threads.

2.5 Relational Databases: SQLite & PostgreSQL

The application implements a hybrid relational database architecture configured with SQL Alchemy ORM. During local development, the system defaults to an offline SQLite file (`interview_system.db`). SQLite provides zero-configuration storage, making local setup, testing, and evaluation completely self-contained. The database maps schemas for candidate profiles, parsed resumes, proctor violations, and detailed interview session scores.

For production environments, the database URL is overridden to connect to a cloud PostgreSQL database instance hosted on Neon. To bridge these environments, the project includes an ETL script (`migrate_to_postgres.py`) and a real-time dual database sync write handler. This infrastructure allows developers to collect mock candidate sessions locally on SQLite, and securely migrate the records into postgres production databases without sequence conflicts, ensuring high data reliability.

2.6 Speech-to-Text AI Engine: OpenAI Whisper & Web Speech API

Transcribing audio answers is key to automated grading. The system utilizes a dual-engine speech-to-text pipeline. In production, the main engine is **OpenAI Whisper**, running a local 'base' model on containerized workers. Whisper uses a sequence-to-sequence encoder-decoder structure to transcribe audio clips of candidate answers with extremely high word accuracy, handling diverse accents and background technical terminology.

To maintain reliability in resource-constrained environments (like Render web servers where Whisper can trigger Out of Memory errors), the system implements a client-side transcription fallback using the browser's native **Web Speech API**. This fallback captures the candidate's speech in real-time, processes transcription on the client machine, and uploads the compiled text directly to the API server, ensuring uninterrupted grading under all hosting configurations.

2.7 Background Task Queue: Redis & Celery

Converting video recordings, running Whisper speech-to-text models, and calling LLM grading APIs are heavy computational operations. Executing these tasks directly inside FastAPI route handlers would lock the request threads, causing candidate screens to freeze and time out. To solve this bottleneck, the production container stack leverages **Celery** as an asynchronous task queue and **Redis** as the task broker.

When an answer is submitted, Uvicorn writes the audio file, pushes a task signature to Redis, and returns a 'processing' status to the client. An isolated Celery worker container pulls the task, runs Whisper offline to extract the transcript text, coordinates with the AI grader to compute scores, and updates the database row. This asynchronous model keeps the main API server highly responsive and stable.

CHAPTER 3: SYSTEM REQUIREMENTS ANALYSIS & DESIGN

3.1 Software Requirements Specification

A rigorous requirements analysis was conducted to establish the operational constraints of the Advanced AI Interview System. The Software Requirements Specification (SRS) defines the hardware, software, and runtime environments needed for development and production. The target environment utilizes standard web browsers for candidates and cloud container stacks for backend operations.

The minimum development requirements include: (1) Python 3.10+ runtime, (2) Git CLI for version control, (3) SQLite 3, (4) Docker Desktop for orchestrating PostgreSQL, Celery, and Redis containers. The minimum client hardware includes a dual-core CPU with at least 4GB RAM, a working 720p webcam, a functional microphone, and a chromium-based browser (Chrome, Edge, or Opera) that supports WebRTC media streams and TensorFlow Face-API landmark model execution.

3.2 Functional Requirements: Candidate Flow

The system's functional requirements map the candidate's journey through the interactive mock interview simulator. Candidate-facing workflows include three primary stages: Registration/Login, Room Calibration, and Active Assessment. The candidate begins by registering credentials on the homepage. Upon logging in, the portal prompts the user to upload a resume in PDF format.

Once the resume is uploaded, the parser extracts technical skills and redirects the candidate to the Calibration Screen. Here, the browser initializes the webcam, runs Face-API, and prompts the user to look straight to align facial landmark vectors. Once calibrated, the candidate enters the simulator room. The system presents questions based on the parsed skills, captures microphone voice bytes, renders real-time transcriptions, and tracks tab blurs.

3.3 Functional Requirements: Recruiter Flow

Recruiters and HR managers interact with the system through a secure administrator dashboard (`/manager.html`). The administrator logs in using pre-seeded secure credentials to access candidate lists. The dashboard provides three main capabilities: Candidate Dossier Review, Integrity Monitoring, and Access Control.

The dossier review page compiles candidate data, including extracted contact details, resume links, average interview scores, and video recordings. The integrity monitor flags cheating warnings (e.g. eye-gaze drifting or tab blurring) with exact timestamps, helping recruiters detect plagiarism. Lastly, the access control panel allows managers to approve or reject candidates, and revoke or grant active interview tokens.

3.4 Non-Functional Requirements

Beyond functional features, the Advanced AI Interview System meets critical non-functional requirements including Performance, Security, Reliability, and Cost-Efficiency. Performance is optimized by ensuring API endpoints return JSON payloads within 200ms. Asynchronous task queues ensure that Whisper audio transcription runs in the background, preventing request timeouts.

Security is maintained by encrypting passwords, storing database URLs as environment variables, and ensuring webcam video remains in client memory during proctoring. Reliability is ensured through local database fallbacks and local TF-IDF text scoring models, allowing the system to function during API outage events. This setup ensures zero operational downtime or cost penalties, making the platform robust.

3.5 System Architecture & UML Use Case Diagrams

The system architecture follows a decoupled, three-tier model bridging static frontend UI clients with containerized API endpoints. The presentation tier (HTML, CSS, JS) is hosted on Vercel, the application logic tier (FastAPI web server, Celery task workers) is containerized on Render, and the database tier is hosted on cloud Neon PostgreSQL databases.

A UML Use Case Diagram maps the core actors and transactions. The Candidate actor initiates registrations, uploads resume PDFs, triggers proctor face calibration, records answers, and downloads scoring reports. The Recruiter actor manages candidate credentials, reviews dossiers, watches proctor violation alerts, and modifies candidate status. The AI Grader system actor runs transcriptions, processes answers, and updates database records.

3.6 System Data Flow Diagrams

Data Flow Diagrams (DFDs) trace the information flow through the Advanced AI Interview System. In Level-0, the Candidate submits PDF resumes and audio recordings to the platform, and receives grading feedbacks. The Recruiter submits access tokens and status approvals, and receives candidate performance dossiers.

In Level-1, information moves through four core backend processes: (1) Process 1.0 (Resume Plumber) parses files and commits parsed skills to the User table; (2) Process 2.0 (Dynamic Selector) queries the question pool and returns questions to the Simulator; (3) Process 3.0 (Web Speech / Whisper) transcribes audio answers; and (4) Process 4.0 (AI Grader) evaluates answers and commits session scores to the database, notifying the Recruiter's dashboard.

3.7 Database Schema Design: Users Table

The relational database schema is structured to ensure fast query times and maintain referential integrity. The **users** table stores account details, parsed resume information, proctoring integrity warnings, and interview metadata.

Field	Type	Description
id	Integer (PK)	Primary key index.
username	String (Unique)	Unique username identifier.
password	String	Hashed user password.
status	String	Candidate state (Approved/Rejected).
access	String	Active interview privilege flag.
resume_path	String	URL to uploaded resume PDF.
skills	String	Parsed candidate skill list.
email	String	Extracted candidate email.
phone	String	Extracted candidate phone.
integrity_notes	String	Timestamped proctoring alerts.

3.8 Database Schema Design: Sessions Table

The **interview_sessions** table maintains referential integrity by linking candidate answers, audio-video URLs, scores, and AI feedback to the parent `users` table via foreign keys.

Field	Type	Description
id	Integer (PK)	Primary key index.
username	String (FK)	Links session to users.username.
date	String	Interview session timestamp.
question	String	Interview question text.
answer	String	Transcribed answer text.
emotion	String	Dominant candidate emotion.
score	Float	Grade score between 0 and 100.
video_url	String	URL to answer video recording.
evaluation_feedback	String	Detailed text feedback from AI grader.

CHAPTER 4: SYSTEM IMPLEMENTATION DETAILS

4.1 Development Environment Setup Flow

Development begins by preparing a virtual environment and configuring environment variables in a local `.env` file. The project is structured with standard Python layouts, placing core API logic in `main.py` and modular backend scripts inside the `modules/` folder.

The setup commands executed are: (1) `python -m venv venv` to create the isolated environment, (2) activates it using `.\venv\Scripts\activate` on Windows, (3) installs pip packages via `pip install -r requirements.txt`. The `.env` file is populated with `DATABASE_URL`, `CLOUDINARY_CLOUD_NAME`, and `GEMINI_API_KEY`. Running `uvicorn main:app --reload` launches the hot-reloading development server on port 8000.

4.2 Core Application Entrypoint: main.py

The main entry point of the API server is **main.py**, which instantiates the FastAPI app and sets up configuration settings. First, it loads environment variables, creates database tables using SQLAlchemy's `Base.metadata.create_all(bind=engine)`, and registers CORS middlewares to allow communication from any frontend client.

Next, `main.py` defines API routers and route endpoints. This includes `/api/register` and `/api/login` for user authentication, `/api/upload_resume` for file uploads, `/api/analyze_answer` to process recorded candidate answers, `/api/get_all_records` to fetch performance dossiers, and `/api/download_pdf_report` to generate and serve the PDF technical reports on recruiter dashboards.

4.3 Resume Parsing Sub-system: resume_parser.py

When a candidate uploads a resume PDF, the server invokes the parser module `modules/resume_parser.py`. This module uses **pdfplumber** to open the file and extract the raw text content page-by-page, converting the document to a structured string. It then runs regular expression (regex) boundaries to find candidate details.

First, it extracts contact info (email and phone). Next, it matches academic degrees (e.g. B.Tech, M.Tech, BCA, MCA) and engineering branches (e.g. Computer Science, AIML, Data Science) by checking the text against predefined regex patterns. Finally, the parsed skills are cross-referenced with a list of programming languages and databases to build a clean skill list for database storage.

4.4 Technical Question Selector: question_retriever.py & question_generator.py

Selecting relevant, high-quality technical questions is key to conducting an effective candidate mock screening. The system coordinates question selection using `modules/question_generator.py`. This module maintains a structured technical question pool, categorized by programming language (Python, Java, C++, JS, SQL), engineering branch, and difficulty level (fresher vs experienced).

When a candidate enters the simulation room, the selector reads the parsed resume skills and queries the database. To ensure a diverse assessment, the generator selects a combination of technical coding, design, and behavioral questions. It checks the candidate's interview session history to ensure that no question is repeated, providing a fair screening process.

4.5 LLM Answer Grading Logic: answer_analyzer.py

Grading descriptive transcript answers is handled by `modules/answer_analyzer.py`. In normal mode, the script calls the **Gemini 1.5 Flash API** (or OpenAI GPT-4o if configured) using a structured JSON prompt template. The prompt instructs the LLM to grade the answer transcript on four categories: Technical Accuracy, Explanatory Depth, Relevance, and Communication Clarity, returning a score out of 100.

If no API keys are found or network connection drops, the module automatically runs a **local TF-IDF Cosine Similarity grading model**. This fallback computes the cosine similarity vector between the candidate's answer and model answers stored in the question pool. It applies bonuses for word count and adjusts scores based on primary emotions (e.g. +10% for confidence, -5% for fear), ensuring uninterrupted grading.

4.6 Proctoring & Gaze Detection Integration

Real-time proctoring is orchestrated on the client side inside the simulator room page (`simulation.html`). When the candidate activates the webcam, a JavaScript function binds to the browser rendering window. It runs **Face-API.js** models in a canvas loop, detecting face presence, head pose rotation, and eye gaze drift vectors at regular intervals.

If the face-api model detects that the candidate's face is missing, a secondary face enters the webcam view, or the eyes drift away from the screen for more than 3 seconds, the browser triggers a proctor warning. It makes an API post request `/api/log_violation` to log the incident. The browser also binds to `window.onblur` to log whenever the candidate switches browser tabs to search for answers.

4.7 Asynchronous Queue Management: celery_worker.py

Asynchronous task workers are implemented in `celery_worker.py`. When a candidate submits an audio-video answer, the FastAPI route handler saves the incoming file stream to a temporary directory, initializes a Celery task signature, and pushes the payload to the Redis broker queue. This prevents the web request thread from blocking during long-running tasks.

A containerized Celery worker pulls the task from the queue, decodes the audio bytes, and runs the Whisper base model to extract the transcript text. Once transcribed, it invokes the answer analyzer to calculate scores and write feedback. Finally, it uploads the recording to Cloudinary and updates the SQLite database row, ensuring high system responsiveness.

4.8 Dual DB Synchronization Script: migrate_to_postgres.py

To sync local candidate data with production databases, the project provides `migrate_to_postgres.py`. This ETL script connects to the local SQLite database (`interview_system.db`) and the production Neon PostgreSQL database using SQLAlchemy session builders. It extracts, cleans, and loads relational tables between environments.

The script extracts candidate profiles from the SQLite `users` table and session details from the `interview_sessions` table. It deduplicates records by checking for unique usernames, performs batch insertions into PostgreSQL, and aligns serial primary key sequence counters to prevent ID collisions on future insertions, ensuring smooth database migrations.

CHAPTER 5: TESTING, QA & RESULTS

5.1 Quality Assurance Plan & Testing Methodology

Ensuring the reliability and accuracy of automated evaluations is key to deploying the Advanced AI Interview System in corporate recruitment pipelines. A comprehensive Quality Assurance (QA) plan was executed to evaluate the system across three core parameters: Resume Parsing Accuracy, AI Grading Consistency, and Proctoring reliability.

The testing methodology involved running automated unit test suites and manual validation loops. Unit tests verified edge-case resume uploads, testing formatting, missing skills, and qualification branch classifications. LLM evaluations were verified by comparing scores generated by Gemini 1.5 Flash against manual grades given by senior technical evaluators at CDAC Mohali to check for consistency.

5.2 Unit Testing & Boundary Edge Validations

Automated unit tests were written to confirm that the resume parsing module correctly handles boundary inputs, including poorly formatted resume PDFs, missing email addresses, and invalid degree descriptions. Table 5.1 maps the validation check results.

Test Case Description	Expected Output	Result
Resume with missing email / phone	Email = None, Phone = None; parse continues	PASSED
Resume with multiple degree patterns	Extracts highest qualification (e.g. B.Tech)	PASSED
No text in uploaded resume PDF	Graceful catch, prompts manual skill input	PASSED
Resume containing special unicode chars	Cleans non-ASCII, parses plain text	PASSED

5.3 LLM Fallback Similarity and DB Sync Verification

To verify that the offline grading engine is robust, a test script (`scratch/verify_parser.py`) was run with API keys disabled. The script simulated candidate submissions and checked the grading outputs. Under API dropouts, the system successfully called the local TF-IDF Cosine Similarity engine, grading transcripts based on keyword overlap and answer length without raising errors.

Additionally, database verification scripts (`scratch/verify_dual_write.py` and `verify_dual_db.py`) were run to check PostgreSQL and SQLite write concurrency. The scripts simulated simultaneous database insertions, confirming that the dual-write session handles transactional commits, rolls back failed database operations to prevent corruption, and synchronizes sequence indices without primary key conflicts.

5.4 User Acceptance Testing & Dashboard Reports

User Acceptance Testing (UAT) was conducted to verify that recruiter workflows and dashboard charts are highly responsive. Recruiter dashboard pages (`manager.html`) were tested under simulated database loads to measure page load speed, chart rendering, and token revocation latency.

The test results confirm: (1) candidate dossier tables load within 120ms by using optimized database query groupings; (2) Highcharts JS 3D charts render smoothly across different browser window dimensions; and (3) admin actions (approving applications or deleting candidate records) update database records in real-time, validating the platform's suitability for corporate deployments.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion & Trainee Learnings

In conclusion, the six-month semester industrial training at C-DAC Mohali provided hands-on experience in building robust, AI-driven software applications. The developed **Advanced AI Interview System** successfully automates candidate screening by integrating resume parsing, real-time webcam proctoring, Whisper transcription, and Gemini grading models into a single platform.

Key learnings acquired during training include: (1) mastering FastAPI's async event loop; (2) configuring Celery workers and Redis queues to handle heavy background task execution; (3) managing PostgreSQL relational database transactions and synchronizations; and (4) implementing browser proctoring models using Face-API.js. These skills align with modern web engineering practices and prepare the trainee for professional software development roles.

6.2 Future Scope & Technical Enhancements

Although the Advanced AI Interview System is fully functional, several advanced features can be added in future development cycles. First, the proctoring engine can be expanded to run audio analysis models on Celery workers. This would allow the system to detect background voices, keyboards clicks, or secondary speakers, improving cheating detection.

Second, the speech analysis pipeline can be enhanced to compute pitch, tone, and pause indicators to measure candidate confidence and communication clarity. Third, the grading engine can be updated to generate follow-up questions in real-time based on candidate answers, creating a more interactive, conversational interview experience.

REFERENCES & BIBLIOGRAPHY

The following books, papers, and online documentation were referenced during the development of this project:

1. **FastAPI Web Framework Documentation:** Detailed guides on asynchronous routing and dependencies. *https://fastapi.tiangolo.com*

2. **ReportLab PDF Compilation Library User Guide:** Documentation on SimpleDocTemplate, flowables, and custom numbered canvases. *https://www.reportlab.com*

3. **Face-API.js GitHub Repository:** Pre-trained weights and API details for browser TensorFlow landmarks models. *https://github.com/justadudewhohacks/face-api.js*

4. **OpenAI Whisper Model Details:** Research paper and repository details for high-accuracy speech-to-text. *https://github.com/openai/whisper*

5. **SQLAlchemy ORM Documentation:** Relational database sessions, query optimizations, and Postgres drivers. *https://www.sqlalchemy.org*

6. **Celery Asynchronous Task Queue User Guide:** Worker process management and Redis broker setups. *https://docs.celeryq.dev*

APPENDIX A: CODE SNIPPET - main.py ROUTING

The following code snippet shows the FastAPI endpoint logic inside **main.py** that handles incoming resume PDF uploads, calls the parser, and returns dynamic question selections to candidate simulation rooms:

```
@app.post("/api/upload_resume") async def upload_resume( resume:
UploadFile = File(...), db: Session = Depends(get_db), current_user: User
= Depends(get_current_user) ): try: # Save resume PDF to file system
file_bytes = await resume.read() resume_dir = "static/resumes"
os.makedirs(resume_dir, exist_ok=True) file_path =
os.path.join(resume_dir, resume.filename) with open(file_path, "wb") as
f: f.write(file_bytes) # Parse resume and extract skills parsed_data =
parse_resume(file_path) current_user.resume_path = file_path
current_user.skills = parsed_data.get("skills", "") db.commit() # Select
questions based on parsed skills questions =
select_questions_for_user(current_user.skills) return {"status":
"success", "questions": questions} except Exception as e: db.rollback()
raise HTTPException(status_code=500, detail=str(e))
```

APPENDIX B: CODE SNIPPET - migrate_to_postgres.py

The following code snippet shows the database migration logic inside **migrate_to_postgres.py** that transfers candidate and session records from the local SQLite database to the production PostgreSQL instance:

```
def migrate_database(): sqlite_session = SQLiteSessionLocal()
postgres_session = PostgresSessionLocal() try: # Migrate user accounts
sqlite_users = sqlite_session.query(SQLiteUser).all() for s_user in
sqlite_users: exists = postgres_session.query(PostgresUser).filter_by(
username=s_user.username ).first() if not exists: p_user = PostgresUser(
username=s_user.username, password=s_user.password, skills=s_user.skills,
email=s_user.email, phone=s_user.phone ) postgres_session.add(p_user)
postgres_session.commit() print("Database migration completed
successfully.") except Exception as e: postgres_session.rollback()
print(f"Migration failed: {e}") finally: sqlite_session.close()
postgres_session.close()
```

APPENDIX C: SYSTEM INSTALLATION GUIDE & CLI

To deploy the Advanced AI Interview System locally or in production containers, execute the following CLI commands in order:

```
# Step 1: Clone the repository and navigate to root $ cd
advanced_ai_interview_system # Step 2: Initialize virtual environment and
install pip packages $ python -m venv venv $ venv\Scripts\activate $ pip
install -r requirements.txt # Step 3: Run SQLite database migrations and
launch the dev server $ uvicorn main:app --reload --port 8000 # Step 4:
Run production container stack using Docker Compose $ docker-compose up
--build -d # Step 5: Execute database migration from SQLite to Postgres $
python migrate_to_postgres.py
```

End of semester industrial training report.